graphtrace

Regression testing for LangGraph agents — as a pytest plugin

Case study · November 2026
Prepared by Anoop Singh
github.com/devbyanoop/graphtrace

Catching what a "looks right" LangGraph agent hides

Why I built graphtrace, what went wrong building it, and what it found once I pointed it at two real, official open-source agents instead of my own demo.

OPEN SOURCE · APACHE 2.0 | LIVE ON PyPI | 222 TESTS PASSING | VERIFIED ON 2 REAL EXTERNAL APPS

Why I built this

Most agent frameworks make it easy to check whether the final answer looks right, and hard to check whether the agent got there safely. A LangGraph agent with branches, retries and tool calls can pass every day while a retry quietly starts firing on every call, a tool stops being invoked, or an approval step gets routed around — none of which shows up if a test only checks the text that comes out at the end.

I wanted to test the path itself, not just the destination: which nodes ran, in what order, what each one changed, what it called — asserted directly, the way a web framework lets you check a response's status code and not just the rendered page. And I wanted it proven on real agents, not a demo built to flatter the tool, so once the core worked I pointed it at two real open-source LangGraph projects and let them find what my own test fixtures couldn't.

222	12	25	4
TESTS PASSING	ASSERTION FUNCTIONS	REAL-AGENT SCENARIOS	REAL BUGS FOUND IN REAL AGENTS

What I set out to do

- **Record a full run** — every node, state change, tool call and LLM call — without touching the agent's own code.
- **A small set of readable assertions**, not a dump of raw trace data to dig through by hand.
- **Replay with zero real LLM calls**, so CI never pays for, or waits on, a real model to pass.
- **An LLM-as-judge** for what a plain assertion can't cover, calibrated against real human labels first — never trusted blindly.
- **Validate it against real external agents** before calling it done.

How it's put together

- **Framework-agnostic core.** The trajectory model and assertion API know nothing about LangGraph specifically — built against a plain data structure, so this isn't locked to one framework.
- **No changes to the agent under test.** Recording works by patching the framework's own calls for the duration of a run, undone automatically when it ends.
- **Record once, replay for free.** A run is written to a plain YAML file and replayed from it afterward — no real network calls, no bill, on every CI run after the first recording.

- **Real numbers, not estimates.** Cost and latency come from what was actually recorded.
- **Judge calibration before trust.** An LLM-as-judge verdict is scored against a human-labelled set with Cohen's kappa before it's relied on for anything.

Problems I ran into

- **A concurrency assumption that didn't hold.** The first design tracked "which node is running" with a thread-local variable. LangGraph runs parallel nodes on worker threads that don't inherit it. Fixed by reading the node/task id LangGraph already attaches to every call.
- **Patching the wrong method.** Intercepting chat-model calls meant patching a base-class method. Every real chat model overrides that method with its own version, so the patch caught nothing. Fixed by patching the actual methods every real model defines.
- **A YAML feature that broke LangGraph's own state.** Round-trip YAML mode wraps numbers in a type that preserves formatting — which LangGraph's checkpointer rejected the moment a recorded value replayed back into a real run. Fixed by converting back to plain types right before a value re-enters real code.
- **A statistic I had wrong.** Cohen's kappa doesn't score 0.0 for "complete disagreement" — a systematic reversal scores -1.0, a real negative correlation and a far stronger signal than "no agreement at all." Corrected the calibration report to reflect that.
- **A feature that was designed but never built.** The data model had room for sub-agent enter/exit steps from week one; nothing produced them, because nothing tested a node that's itself another graph. Building it surfaced a worse bug: state tracking was shared globally instead of scoped per sub-agent, so two sub-agents running at once could corrupt each other's recorded data. Fixed by scoping it per sub-agent.

What it found once I pointed it at real agents

Once the core held up against my own demo agent, I ran it against two real, official open-source LangGraph projects — langgraph-swarm-py and deepagents — across 25 scenarios built from their own test suites, not invented ones.

PROJECT	WHAT IT FOUND
langgraph-swarm-py	Tool calls attributed to the wrong node — a newer LangChain agent abstraction had stopped passing an explicit config the recorder relied on.
langgraph-swarm-py	Sub-agent enter/exit markers recorded in the wrong order.
langgraph-swarm-py	A real agent-to-agent handoff crashed recording outright — the handoff object wasn't safe to write to YAML.
deepagents	A tool's runtime object leaked internal state (object ids, open file handles) into recordings, so the same logical call hashed differently every run.

One real limitation surfaced too, not a bug: a tool that internally calls another LangGraph agent loses that nested agent's step-by-step detail on replay — the final answer is still correct, just not the path that produced it. Documented rather than hidden.

What it looks like now

- 222 tests pass — the core recording/replay/assertion/judge/report pipeline, plus the 25 real-agent scenarios above.
- Published on PyPI. `pip install graphtrace` registers it as a pytest plugin automatically.
- A CLI turns a real exported production trace into a generated test; a static HTML report gives a plain pass/fail verdict for anyone who isn't the engineer who wrote the test.
- CI runs the full suite and a strict documentation build on every push.

If you're running an agent like this

Wrong node attribution, steps recorded out of order, a crash on a real handoff, a cache key that silently broke between runs — four real bugs, found in real, official open-source projects, and every one of them came from a system that already had passing tests.

That's exactly the kind of gap that stays invisible until someone checks the actual path, not just the output. If you're running a LangGraph agent — or anything with the same branches/retries/tool-calls shape — in production, or about to put one there, it's worth asking whether your current tests would catch any of that. Most wouldn't. They're built to check the answer, not the route it took to get there.

graphtrace is open source and ready to point at your own graph, with no changes needed to the agent itself.

I'm available for freelance work in test automation and AI-agent testing — from framework design to hardening an existing agent for production.

Code & full build history: **github.com/devbyanoop/graphtrace** · Package: **pypi.org/project/graphtrace** · Contact: **github.com/devbyanoop**

graphtrace is open source under the Apache License 2.0. This case study describes graphtrace v0.1.0.